

Server-side React and Next.js



Roland Guijt

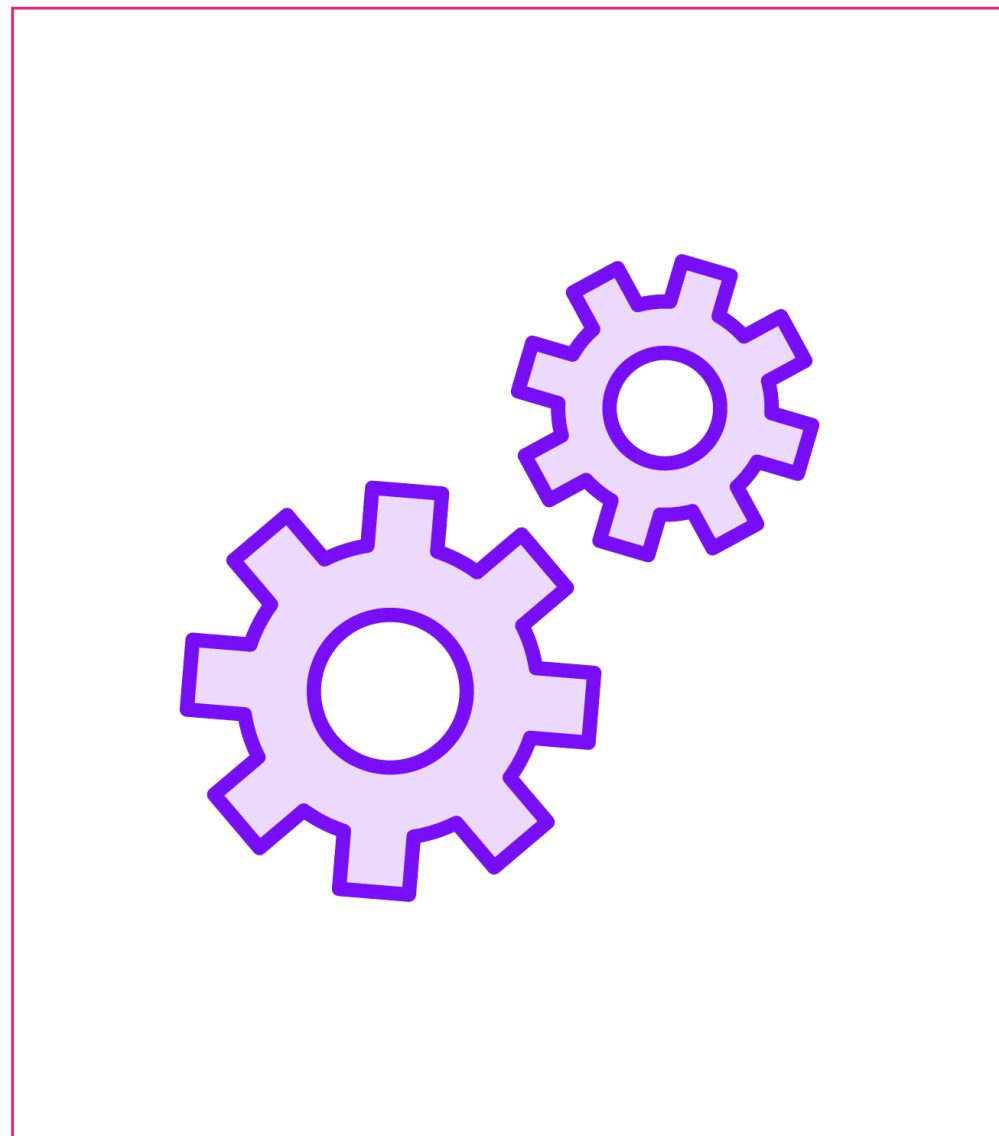
Freelance consultant and trainer

@rolandguijt roland.guijt@gmail.com

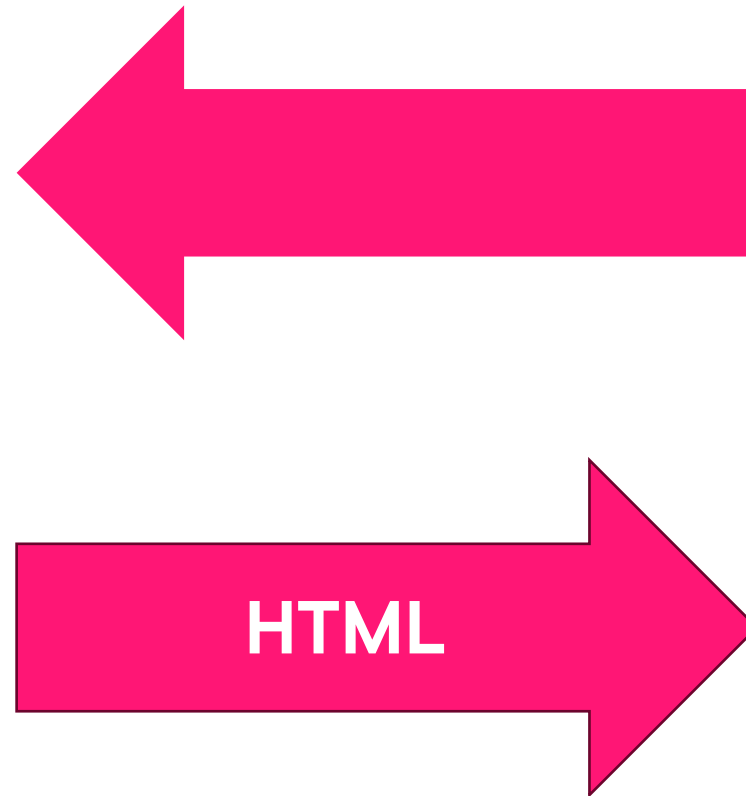


Traditional Web Application

Server



Browser



Server-rendered Applications

Caching

Secrets

Libraries

Feels faster

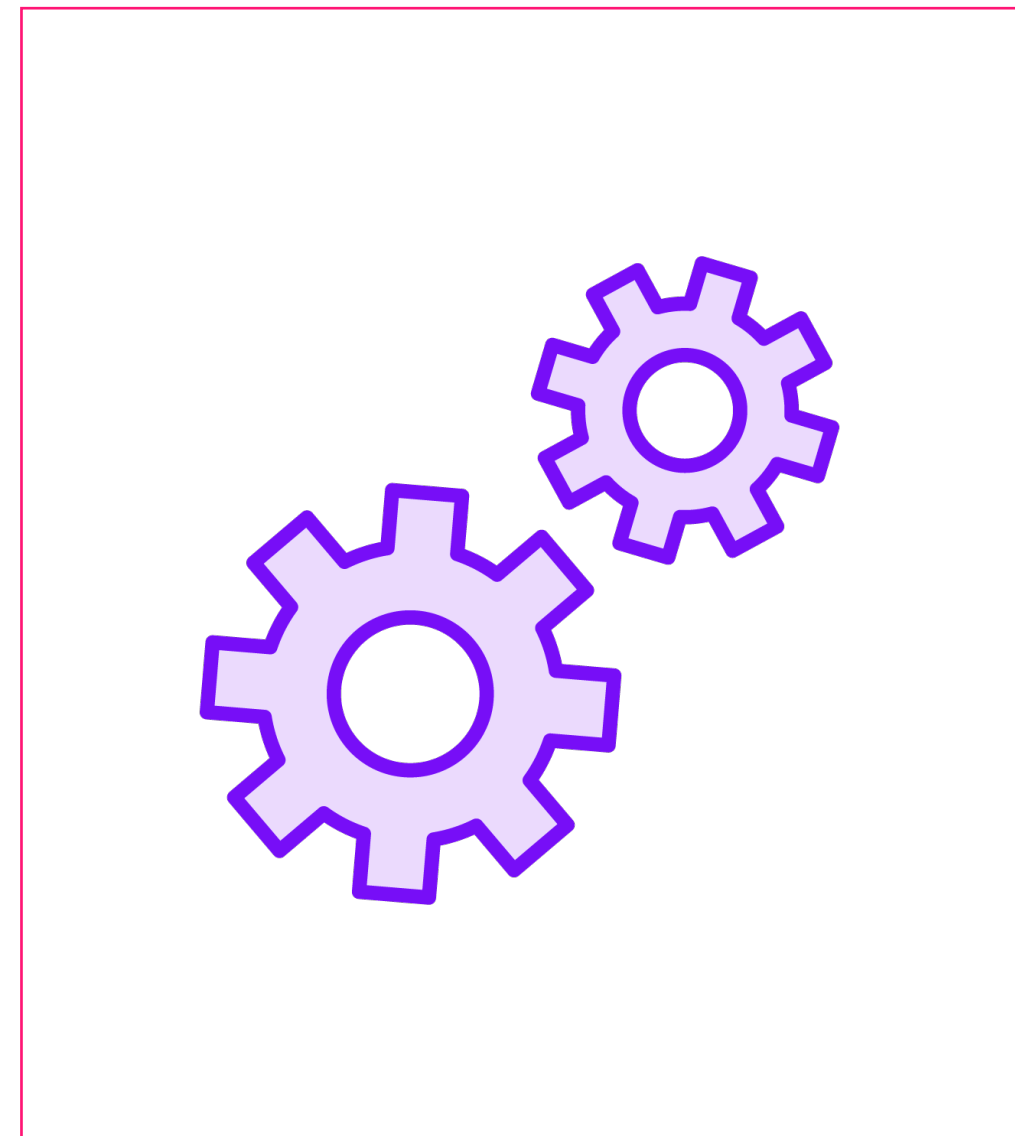


Single Page Application (SPA)

Server



Browser



Single Page Applications

Smooth UX

No server

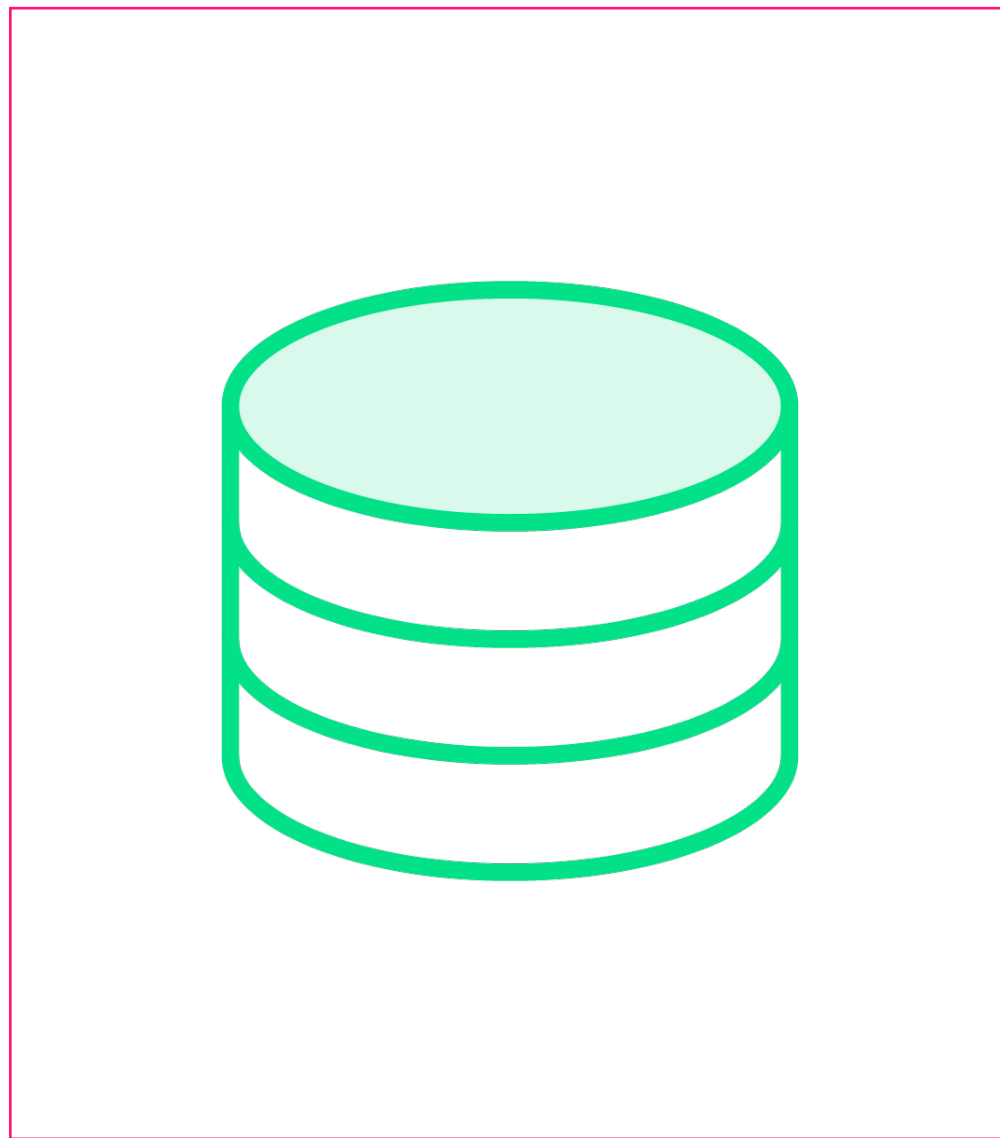
Scalability

Offline possibilities

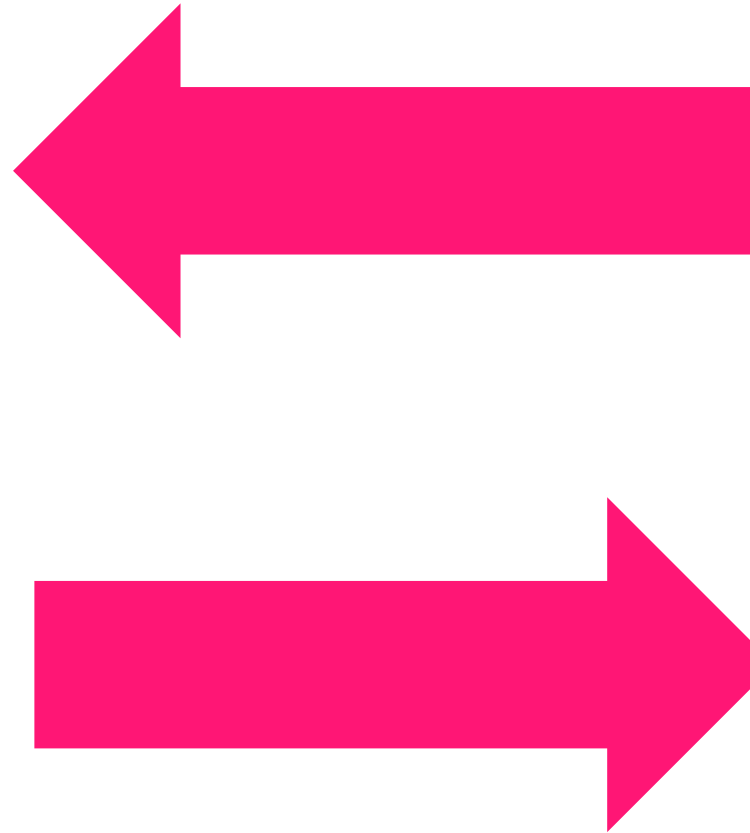
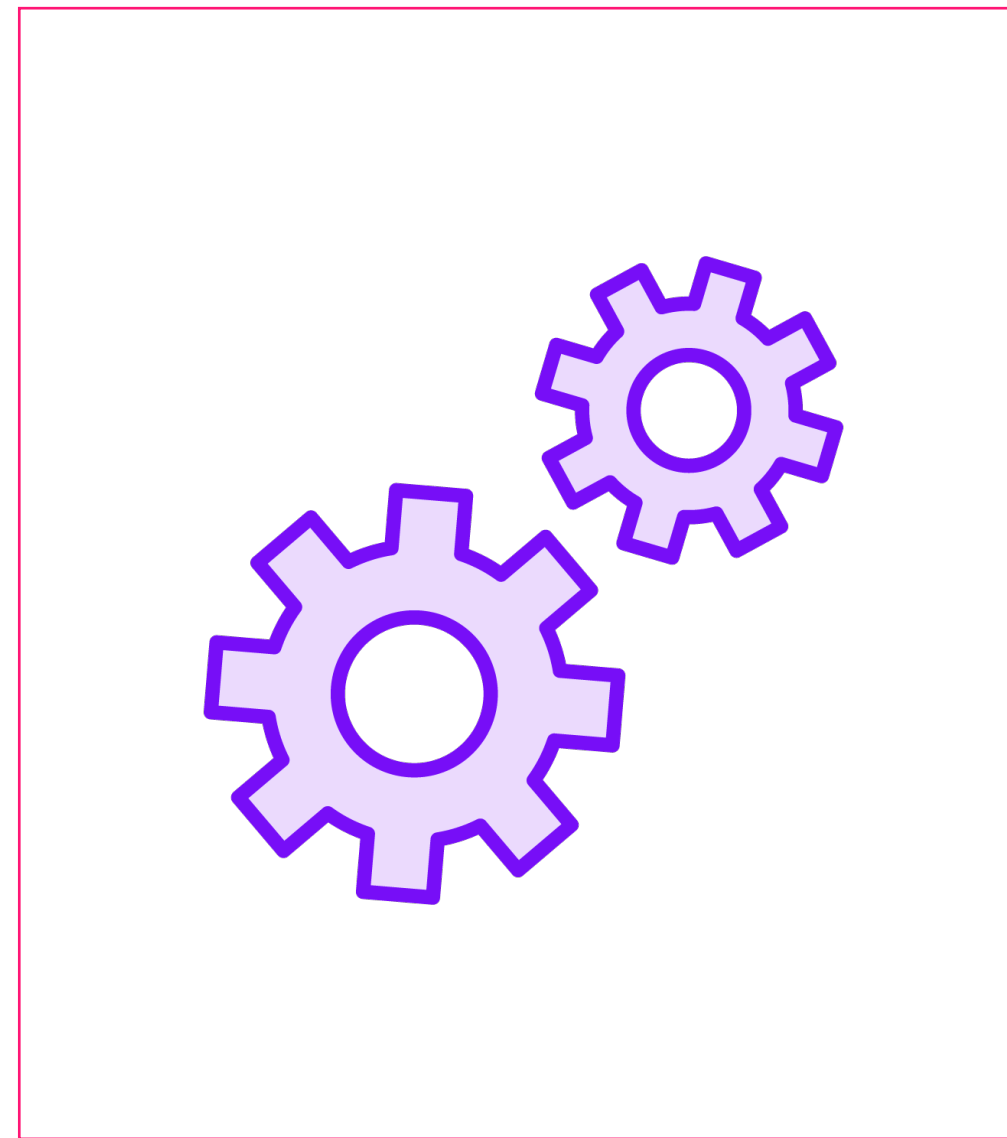


Single Page Application (SPA)

API



Browser



Vite and Server-side features

No support

Pure client-side development

Server-side features add complexity

Application is split in two



Next.js

Both client- and server-side React development

Vercel

Tools and starting template

Supports server-side components and actions

Hosting options

More opinionated

Learn basics with Vite first



**React client-side in
combination with
server-side components and
actions:**

The best of both worlds



**The server-based Next.js
router is called App Router**



Server Components in Next.js

Every page is a server component
Node.js runs javascript and renders jsx



**Next.js uses streaming to
split up the rendering**



**Server components don't
support interactivity**



Nesting Client and Server Components

```
<ServerComponent>  
  <ClientComponent />  
</ServerComponent>
```

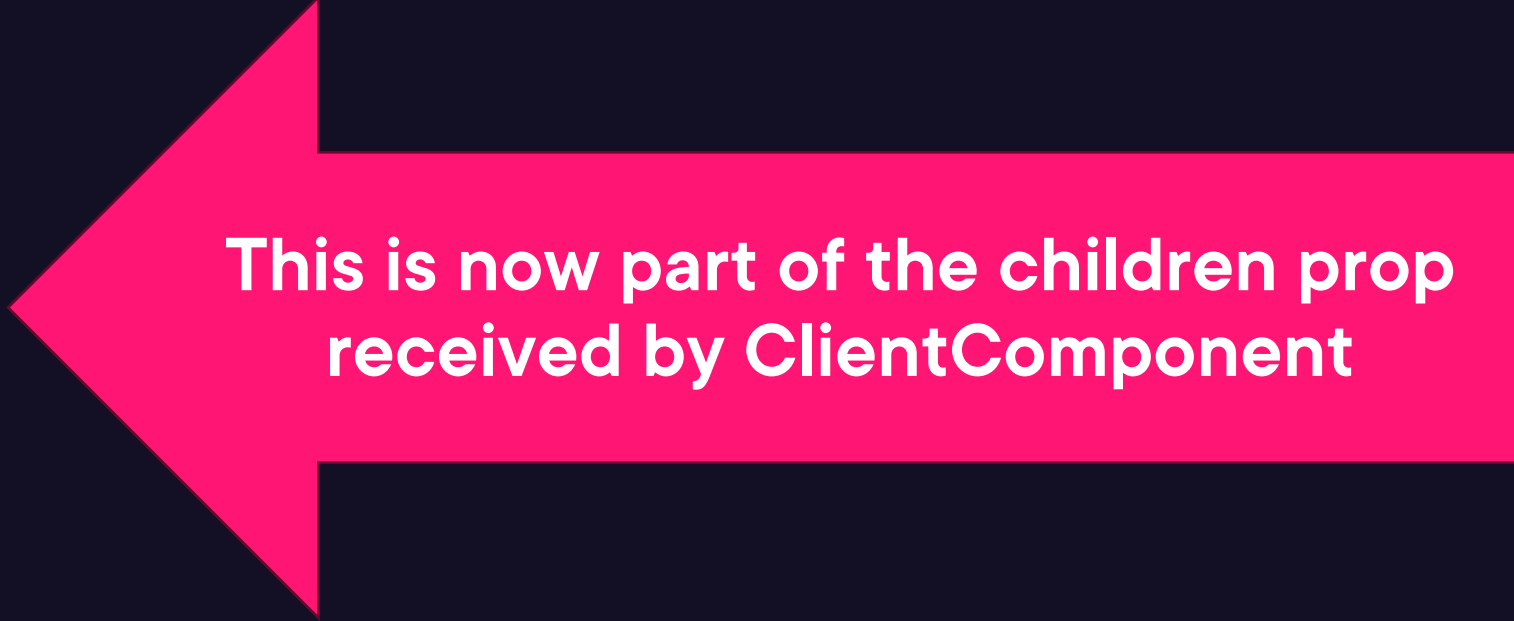


All components rendered by
ClientComponent are also client
components



Nesting Client and Server Components

```
<ServerComponent>  
  <ClientComponent>  
    <ServerComponent />  
  </ClientComponent>  
</ServerComponent>
```



This is now part of the children prop
received by ClientComponent



React Server Component Payload



use

Introduced in React 19

Pure client-side applications need
an external library

Unless the server creates the promise



Using Server Components and Actions

Add complexity

But at the same time less and more
straightforward code

More in-depth courses available



Next up:
Application Design

